ROADWATCH · OXHACK

Technical Manual

Architecture, data, methods, verified results, limitations and roadmap

Contents

1. Overview 2
2. System architecture and project structure 2
3. Data sources, coverage, ingestion and provenance 4
   3.1 Sources 4
   3.2 Ingestion and validation [Implemented] 4
   3.3 Reproducibility [Measured] 5
4. Geographic matching, ambiguous reports and unknown labels 5
   4.1 Matching [Implemented] 5
   4.2 Two targets [Implemented] 5
   4.3 Unknown labels [Implemented] 6
5. Features and leakage prevention 6
   5.1 Features [Implemented] 6
   5.2 Point-in-time rules [Implemented] 6
6. Models, baselines, calibration and walk-forward backtesting 7
   6.1 Models [Implemented] 7
   6.2 Selection without test data [Implemented] 7
   6.3 Walk-forward test [Implemented] 8
7. Evaluation metrics and results 8
   7.1 Metrics 8
   7.2 Results: strict single-block target, 67 test weeks [Measured] 8
8. Inspection-list generation and capacity constraints 10
9. The four capture definitions and inspection-coverage assumptions 10
10. The March 2026 surge and weekly volume forecasts 13
11. Real-life operation 14
12. Improvement roadmap [Proposed] 14
13. Reproduction, environment, testing and troubleshooting 15
   13.1 Environment 15
   13.2 Reproduce the results 16
   13.3 Tests 16
   13.4 Troubleshooting 16
14. References 17
15. Glossary 17

1. Overview

RoadWatch ranks Brooklyn's **23,448** eligible street blocks by how likely each is to receive a **311 pothole report in the next seven days**. It then turns the ranking into a weekly inspection worklist of fixed size. Everything is evaluated retrospectively on public data. RoadWatch is a decision-support prototype, not a live municipal system.

This manual labels every statement so that implementation, evidence, assumptions and plans are never confused:

Label	Meaning
[Implemented]	Code that exists in this repository and is covered by tests
[Measured]	A result read from a saved evaluation file (paths given)
[Assumption]	Something the results depend on that the data cannot confirm
[Proposed]	A future improvement; not built, not measured

What RoadWatch does **not** claim:

- It does not predict confirmed physical defects.
- It does not prevent failures or save money; none of these outcomes has been measured.
- The prediction target is a **public report**.

Authority:

- Where older stage reports differ, the Stage 7 audit (`docs/stage7_audit_report.md`) and its corrections take precedence.
- The default inspection list is the **strict single-block LightGBM top-K** list (`config/stage5.json` → `dashboard_default_list = strict_topk`).

2. System architecture and project structure

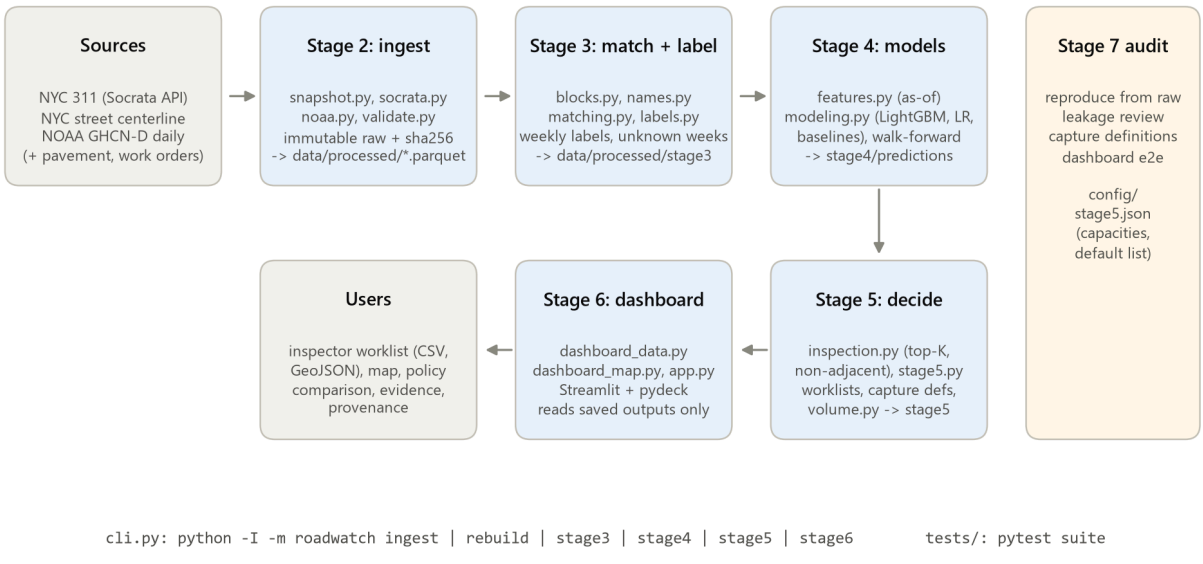


Figure 1. Stages and modules. Each stage writes immutable outputs that the next stage reads; the dashboard only reads saved outputs.

Path	Contents
`src/roadwatch/`	Python package. Stage 2: `sources.py`, `socrata.py`, `noaa.py`, `snapshot.py`, `normalize.py`, `validate.py`, `pipeline.py`. Stage 3: `names.py`, `blocks.py`, `matching.py`, `labels.py`, `stage3.py`. Stage 4: `features.py`, `modeling.py`, `evaluation.py`, `stage4.py`. Stage 5: `inspection.py`, `volume.py`, `stage5.py`. Dashboard: `dashboard_data.py`, `dashboard_map.py`. CLI: `cli.py`
`app.py`	Streamlit dashboard (rendering only; logic lives in `dashboard_data.py`)
`config/stage5.json`	Capacities, policies, tie-break, ZIP cap, volume-model settings, default dashboard list
`data/raw/`	Immutable gzip snapshots of exact API responses plus `MANIFEST.jsonl` (git-ignored)
`data/processed/`	Validated Parquet tables and stage outputs: blocks, labels, features, predictions, worklists (git-ignored)
`outputs/`	Evaluation tables and JSON (`stage3` … `stage7`), dashboard inputs (`stage6`), final deliverables (`final`)
`tests/`	pytest suite (synthetic fixtures for logic; integration checks on saved outputs)
`scripts/`	Stage 1 audit scripts, the Stage 7 reproduction and browser checks, and `scripts/final/` (deliverable builders and verifiers)
`docs/`	Stage reports 2–7, data audit, risks, and `docs/final/` (this manual, user guide, speech, demo, run-of-show)

[Implemented] Command line, run as `python -I -m roadwatch <command>`:

Command	Purpose
`ingest <sources>`	Download or resume, validate, finalise and promote
`rebuild`	Offline, from archived snapshots
`status`	List snapshots
`report`	Write the data-quality report
`stage3`	Block index, matching, weekly labels
`stage4 features`, `select`, `test`, `evaluate`, `uncertainty`	Modelling
`stage5 …`	Decision engine, capture definitions, volume forecast, single-week worklists
`stage6 build`	Precompute dashboard inputs

3. Data sources, coverage, ingestion and provenance

3.1 Sources

Source (NYC Open Data id)	Role	Rows (processed)	Observed range	Used by the model?
311 Service Requests, Street Condition, Brooklyn (`erm2-nwe9`)	primary	134,727	2020-01-01 → 2026-10-23	yes (pothole descriptor → labels and history features)
311 Street Condition with missing borough, citywide (`erm2-nwe9`)	supplementary	5,768	2020-01-01 → 2026-10-18	only reports recovered to Brooklyn
Street Centerline (CSCL), Brooklyn (`inkn-q76z`)	reference	27,673	static 2026 snapshot	yes (blocks and static road attributes)
NOAA GHCN-Daily, Central Park (`USW00094728`)	weather	2,577	2019-10-01 → 2026-10-20	tested; excluded by validation-only selection
Street Pavement Ratings (`6yyb-pb25`)	conditional	135,776	2007-06-01 → 2026-09-30	no (diagnostics only)
Pothole Work Orders, closed (`x9wy-ing4`)	conditional	110,830	2004-02-08 → 2026-09-28	no (diagnostics only)
In-house resurfacing (`ffaf-8mrv`)	conditional	88,921	2002-10-01 → 2026-10-14	no
Resurfacing schedule (`xnfm-u3k5`)	conditional	108,587	2019-11-11 → 2026-10-30	no

Coverage notes:

- **Retrieval.** All snapshots were retrieved on 2026-10-24.
- **Pothole reports.** The relevant 311 subset is *Street Condition / Pothole*: **71,609** Brooklyn reports over **355** weekly forecast origins, 2020-01-06 → 2026-10-19.
- **Block universe. 23,448** street blocks covering **2,617 km**, with a median length of **82** m.

3.2 Ingestion and validation [Implemented]

1. **Paging and completeness.** Socrata sources are paged by `:id`. A partition is accepted only when the rows fetched, the unique ids and the server's `count(*)` agree. The originally supplied 311 CSV had been truncated by a server error, which is why the API is used.
2. **Immutable archive.** Each response is stored exactly as received (gzip) with a sha256 checksum in the snapshot manifest. Snapshots are never edited.
3. **Normalisation.** Original fields are kept; typed columns are added. Floating timestamps are parsed without time-zone conversion. Exact duplicates are dropped; for repeated ids, the latest `:updated_at` is kept.
4. **Promotion.** Validation checks schema, coverage, id uniqueness, timestamp formats and missingness. A processed table is replaced only when a complete snapshot has no FAIL, so a failed refresh leaves the previous table untouched.
5. **Data-source status.** The *Data provenance* view of the dashboard lists each source's latest snapshot, retrieval time, rows, observed range and validation status (pass or warn).

3.3 Reproducibility [Measured]

`scripts/audit/stage7_reproduce.py` rebuilt everything offline from the archived snapshots into a scratch directory (`outputs/stage7/reproducibility.json`):

- **8/8** processed tables, **18/18** Stage 3 outputs and **6/6** feature files are byte-identical.
- Refitting the frozen LightGBM at two retrain dates reproduced the saved predictions with a maximum difference of **0.0**.

4. Geographic matching, ambiguous reports and unknown labels

4.1 Matching [Implemented]

`matching.py` places each report on CSCL blocks hierarchically:

1. **Address and coordinates:** a coordinate snap validated by the street name.
2. **Street-name matching:** space-insensitive normalised keys (for example MC DONALD / MCDONALD), plus 5 coordinate-validated aliases.
3. **Intersections:** the CSCL node shared by the two named streets.
4. **Blockface descriptions:** the segments between the two cross streets.

Reports whose description fits several blocks keep **all** candidate blocks.

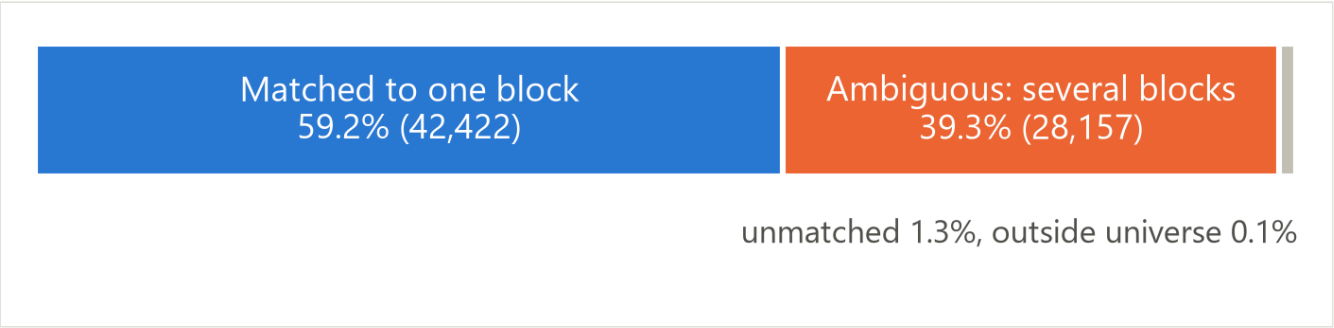


Figure 2. Matching outcome for all 71,609 primary reports (`outputs/stage3/matching_diagnostics.json`).

[Measured]

- **Overall:** 59.2% of reports match one block, 39.3% are ambiguous, 1.3% are unmatched and 0.1% fall outside the block universe.
- **Ambiguous reports:** of the 28,157, most name an intersection (**23,824**) and **4,083** are blockface descriptions. The median ambiguous report has **4** candidate blocks.
- **Test period:** 44% of the 16,855 reports are ambiguous, comprising 9,379 unique, 5,717 intersection reports whose candidates meet at one node, and 1,759 other ambiguous reports.

4.2 Two targets [Implemented]

Target	Definition	Test prevalence
Strict single-block (`primary`, default)	Each report counts on exactly one block: its unique match or, for ambiguous reports, a deterministic pseudo-random pick (a sha256 hash of a salt, the report id and the block id)	0.9% of block-weeks

Target	Definition	Test prevalence
Multi-block (`multi_block`)	Every candidate block of a report counts as positive	1.7%

The multi-block target is easier: a block is positive if it is *one of the possible* locations. Its precision (16.4%) is therefore not comparable with the strict target's 9.5%.

4.3 Unknown labels [Implemented]

Some pothole requests have no usable borough or street:

- **Recovery: 5,567** supplementary requests were inspected and **111** were recovered with Brooklyn evidence.
- **Unknown weeks:** if the potentially hidden Brooklyn share in a week exceeds **5%**, that week is labelled **unknown**. This applies to **23** of 355 weeks, plus the incomplete final week.
- **Unknown is never zero:** unknown block-weeks are excluded from training and evaluation and never treated as "no report". Of the strict target's **8.3 million** block-weeks, **61,313** are positive and **558,649** unknown.

5. Features and leakage prevention

5.1 Features [Implemented]

The selected feature set (`history_static`, **38** features; `outputs/stage4/feature_definitions.md`) has two groups.

History features:

- fractional and any-candidate report counts over 7, 30, 90 and 365 days;
- exponentially decayed counts with half-lives of 7, 30 and 90 days;
- days since the last report, and a no-report-in-365-days flag;
- the trend between the 90-day and 365-day rates;
- matured-week positive rate over 52 weeks;
- neighbouring-block and 2,000 ft grid-cell counts;
- Brooklyn-wide recent volume and week-of-year terms.

Static features: length, travel, parking and total lanes, street width, posted speed, truck route, bike lane, one-way, adjacent-block count, street type and snow priority.

Excluded by selection:

- **Weather** (freeze–thaw days, precipitation, snow) was tested but excluded by validation-only selection. It degraded LightGBM on validation, where weekly weather values act as week identifiers.
- **Pavement ratings and work orders** stay experimental.

5.2 Point-in-time rules [Implemented]

Input	Available to a forecast on Monday *t* if…
311 reports	created before *t* − 2 days (publication lag)

Input	Available to a forecast on Monday *t* if…
NOAA weather	dated on or before $t - 5$ days
Weekly label history	the week matured by $t - 14$ days (7-day window plus lag)
Street attributes	static 2026 snapshot **[Assumption]**: attributes did not change materially
Target	at least one report in $[t, t + 7$ days), Monday 00:00 New York time

[Measured] The Stage 7 leakage review made these checks:

- recomputed features at 7 origins;
- confirmed the 311 and NOAA cutoffs are exact;
- made 240 hand recomputations, with 0 mismatches;
- confirmed that no label or id column enters any model;
- checked all 16 walk-forward windows of every experiment.

One minor issue remains. `unlocated_weeks_in_365d` overlaps the 2-day embargo for 11 training origins in early 2021. It is a week-level constant, does not affect validation or test features, and cannot inflate test results.

6. Models, baselines, calibration and walk-forward backtesting

6.1 Models [Implemented]

Model	Description
LightGBM	Gradient-boosted trees: `num_leaves` 15, `min_data_in_leaf` 200, 38 features, deterministic mode, early stopping on a holdout
Logistic regression	L2-regularised (C = 0.01), `history` feature set
Historical-rate baseline	Share of the block's last 52 matured observable weeks with a report, shrunk toward the Brooklyn rate (prior of 2 weeks)
Recency baseline	Decayed report count, 90-day half-life
Random order	Constant score; ties broken by a deterministic hash

Training uses all positives and a deterministic 10% sample of negatives, with weights 1/0.1. Scores are converted to probabilities by Platt scaling on the holdout. Probabilities are used for context only: **rankings drive every decision**.

6.2 Selection without test data [Implemented]

- **Data split:** training origins 2021-01-04 → 2024-12-30; validation 2025-01-06 → 2025-06-30.
- **Rule:** hyper-parameters, feature set and sampling rate were chosen by validation average precision.
- **Frozen before testing:** the record `data/processed/stage4/selection.json` was frozen at 18:49:49 UTC on 2026-10-24, before the first test prediction at 18:54:08 UTC.

6.3 Walk-forward test [Implemented]

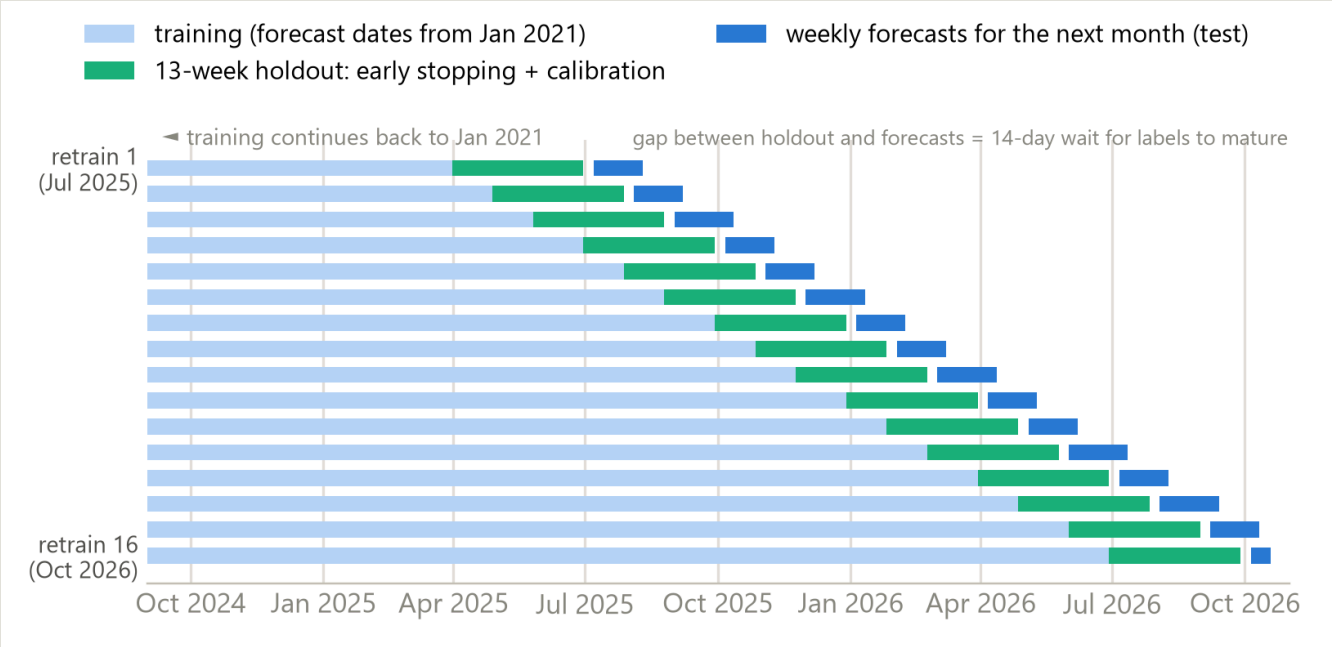


Figure 3. The 16 monthly retrains of the strict LightGBM (`data/processed/stage4/models/primary__LGBM/metadata.json`). Each fit uses forecast dates from January 2021 up to the holdout; the last 13 matured weeks are the holdout for early stopping and calibration; the model then ranks blocks for each week until the next retrain.

The test covers **67** weekly forecasts (2025-07-07 → 2026-10-12) with **16** monthly retrains. Labels need 14 days to mature, so each retrain on date *r* only uses forecast dates up to *r* − 14 days.

[Assumption] The same 13-week holdout is used for both early stopping and calibration. This can make calibration plots slightly optimistic. Rankings are unaffected, because Platt scaling is monotone.

7. Evaluation metrics and results

7.1 Metrics

Metric	Definition
Precision@1%	Share of the 235 listed blocks that received at least one report in the week (against the list's own target labels)
Report capture	Share of next-week unique reports that fell on listed blocks. Four variants are defined in §9
Average precision (AP)	Area under the precision–recall curve over all blocks
Calibration in the large	Sum of predicted probabilities ÷ observed positives (1 = right volume)
95% CI	Percentile bootstrap over test weeks (1,000 resamples); *paired* when comparing two lists on the same weeks

7.2 Results: strict single-block target, 67 test weeks [Measured]

Model	Precision@1%	Report capture@1%	Average precision
LightGBM	**9.5%**	**13.4%**	0.052

Model	Precision@1%	Report capture@1%	Average precision
Logistic regression	8.8%	12.6%	0.054
Recency baseline	8.1%	11.5%	0.040
Historical-rate baseline	6.6%	8.4%	0.028
Random order	0.9%	1.0%	0.008

Source: `outputs/stage4/test_results.csv`. Paired week-level bootstrap (`outputs/stage4/bootstrap_comparisons.json`), precision@1% difference for LightGBM:

Comparison	95% CI
vs historical rate	**+2.3 to +3.4 pp** (AP +0.020 to +0.029)
vs recency	+0.9 to +1.7 pp
vs logistic regression	+0.4 to +0.9 pp (AP difference −0.005 to +0.001, not significant)

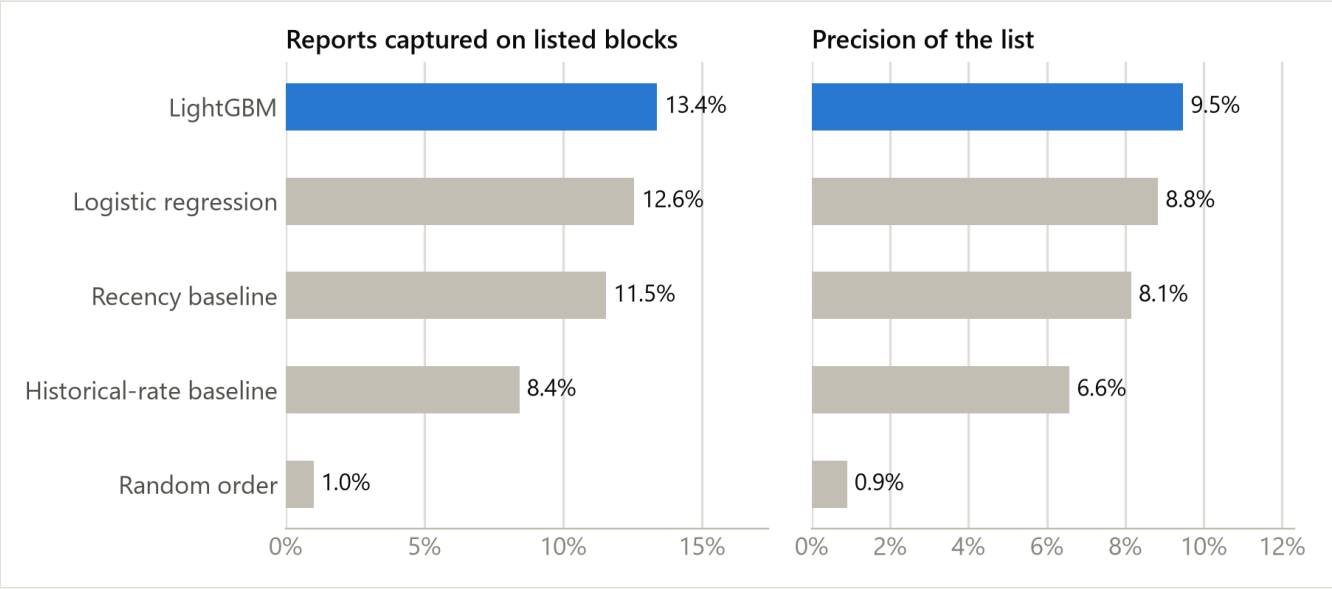


Figure 4. Strict target, top 1% (235 blocks per week), 67 test weeks.

The 95% CIs for capture come from `outputs/stage5/capture_definitions.csv`:

- **LightGBM list:** 13.4% (12.5–14.4).
- **Historical-rate list:** 8.4% (7.7–9.0).

The historical-rate list there was built from the multi-block run of the same baseline. Its score uses only target-independent features, and it shares 15,743 of 15,745 block-weeks with the strict-target list; their capture values are identical.

Multi-block target, for reference only. LightGBM precision is 16.4% against 11.7% for the historical rate (paired CI +3.9 to +5.8 pp), with prevalence 1.7%.

Calibration. Predicted/observed report volume over the test period was 1.05 for the strict LightGBM and 0.99 for the multi-block model. In March 2026 it fell to 0.35 and 0.23 respectively (§10).

8. Inspection-list generation and capacity constraints

[Implemented] (`inspection.py`, `stage5.py`)

1. **Capacity.** K = ceil(share × 23,448): **118 / 235 / 469** blocks for 0.5% / 1% / 2%, or any custom K in the dashboard. Capacity is always chosen by a person; the volume forecast never sets it (§10).
2. **Selection.** Rank all eligible blocks by the week's score and take the top K. Ties are broken by a deterministic splitmix64 hash of (block, week).
3. **Optional rules** (evaluated, not default):
   - **Non-adjacent:** skip a block that shares a CSCL intersection node with a block already selected.
   - **ZIP cap:** at most 3× a ZIP's share of blocks (`--diversify`). It had no measurable effect.
4. **Explanation.** Each listed block gets:
   - a priority tier: P1 = top 20% of the list, P2 = next 30%, P3 = rest;
   - a rationale from its report history;
   - its top three LightGBM feature contributions, presented as supporting evidence, not causes;
   - road attributes, a midpoint and a WGS84 line geometry.
5. **Export.** CSV and GeoJSON with 24 attribute columns: rank, block id, street, ZIP, score, history counts, attributes, rationale, drivers, recommended action, model version and midpoint coordinates.

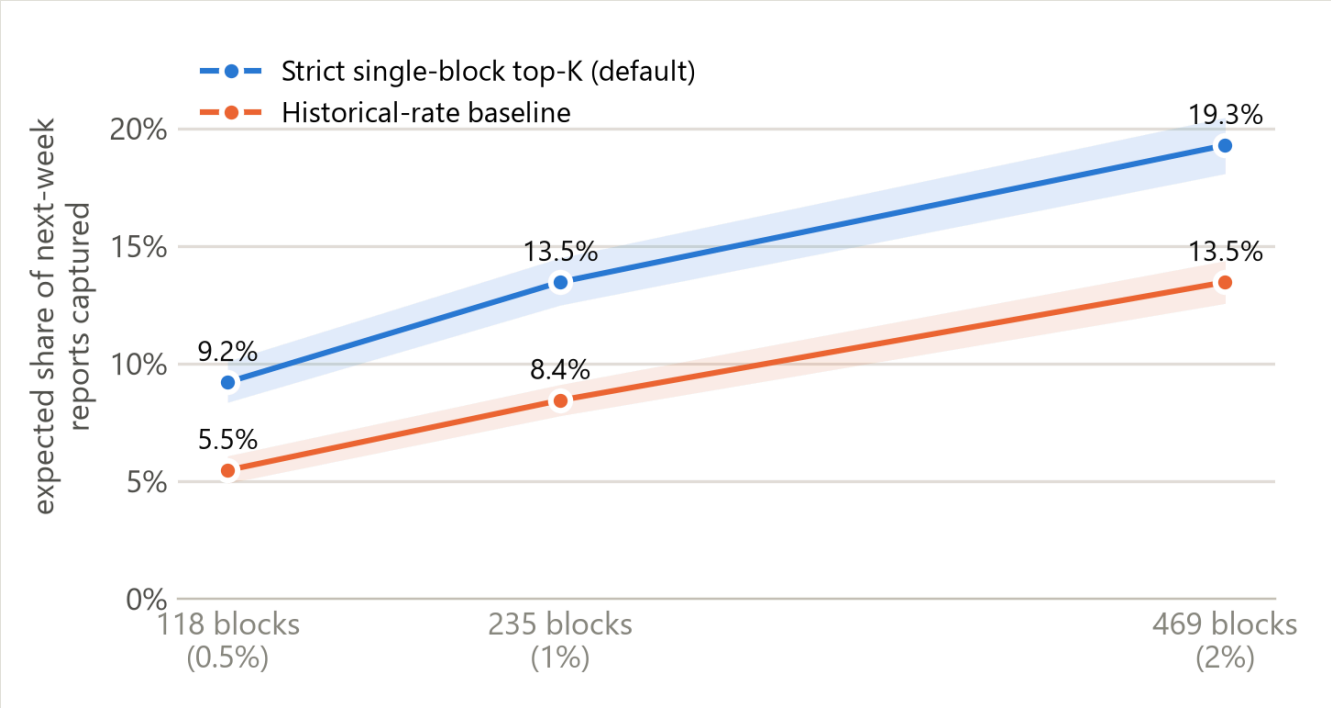


Figure 5. Expected (fractional) share of next-week reports captured by the default list and the historical-rate baseline at the three capacity scenarios, test weeks, with 95% CI bands (`outputs/stage5/capture_definitions.csv`).

[Measured] Expected capture rises from **9.2%** at 118 blocks to **13.5%** at 235 and **19.3%** at 469. The historical-rate baseline reaches 5.5%, 8.4% and 13.5%. The default list at 1% therefore matches the baseline at 2%: half the inspections for the same expected capture, under this metric.

9. The four capture definitions and inspection-coverage assumptions

When a report has several candidate blocks, "did the list catch it?" has more than one honest answer:

Definition	A report counts as captured if…	Assumption
Any candidate	any of its candidate blocks is inspected	optimistic: one inspection covers every candidate
Assigned block	its single assigned (hash-picked) block is inspected	an unbiased estimate of fractional capture
Fractional (expected)	credit = share of its candidate blocks inspected	the true location is equally likely to be any candidate
Intersection-node	unique reports: assigned block. Intersection reports whose candidates meet at one node: any candidate. Other ambiguous reports: fractional	**[Assumption]** inspecting a block also covers its end intersections

[Measured] Unique reports captured at 1% (95% CI), from `outputs/stage5/capture_definitions.csv`.

Test weeks:

List	Fractional	Intersection-node	Any candidate
Strict single-block top-K (default)	**13.5% (12.5–14.5)**	16.4% (15.3–17.5)	17.0% (16.0–18.1)
Strict single-block + non-adjacent	12.3% (11.5–13.2)	17.3% (16.3–18.4)	18.3% (17.2–19.6)
Multi-block + non-adjacent (Stage 5 list)	11.3% (10.6–12.1)	17.3% (16.3–18.4)	18.5% (17.4–19.7)
Plain top-K multi-block	12.8% (11.8–13.9)	15.5% (14.5–16.7)	16.2% (15.2–17.4)
Historical-rate baseline	8.4% (7.8–9.1)	10.8% (10.0–11.6)	11.5% (10.5–12.4)

Validation weeks:

List	Fractional	Intersection-node	Any candidate
Strict single-block top-K (default)	9.7% (8.4–10.9)	13.2% (11.7–14.7)	13.9% (12.5–15.4)
Strict single-block + non-adjacent	9.0% (8.0–10.0)	13.8% (12.4–15.3)	14.7% (13.1–16.1)
Multi-block + non-adjacent (Stage 5 list)	8.3% (7.4–9.2)	13.9% (12.5–15.3)	15.0% (13.4–16.4)
Plain top-K multi-block	8.6% (7.4–9.5)	11.6% (10.2–12.9)	12.3% (10.7–13.6)

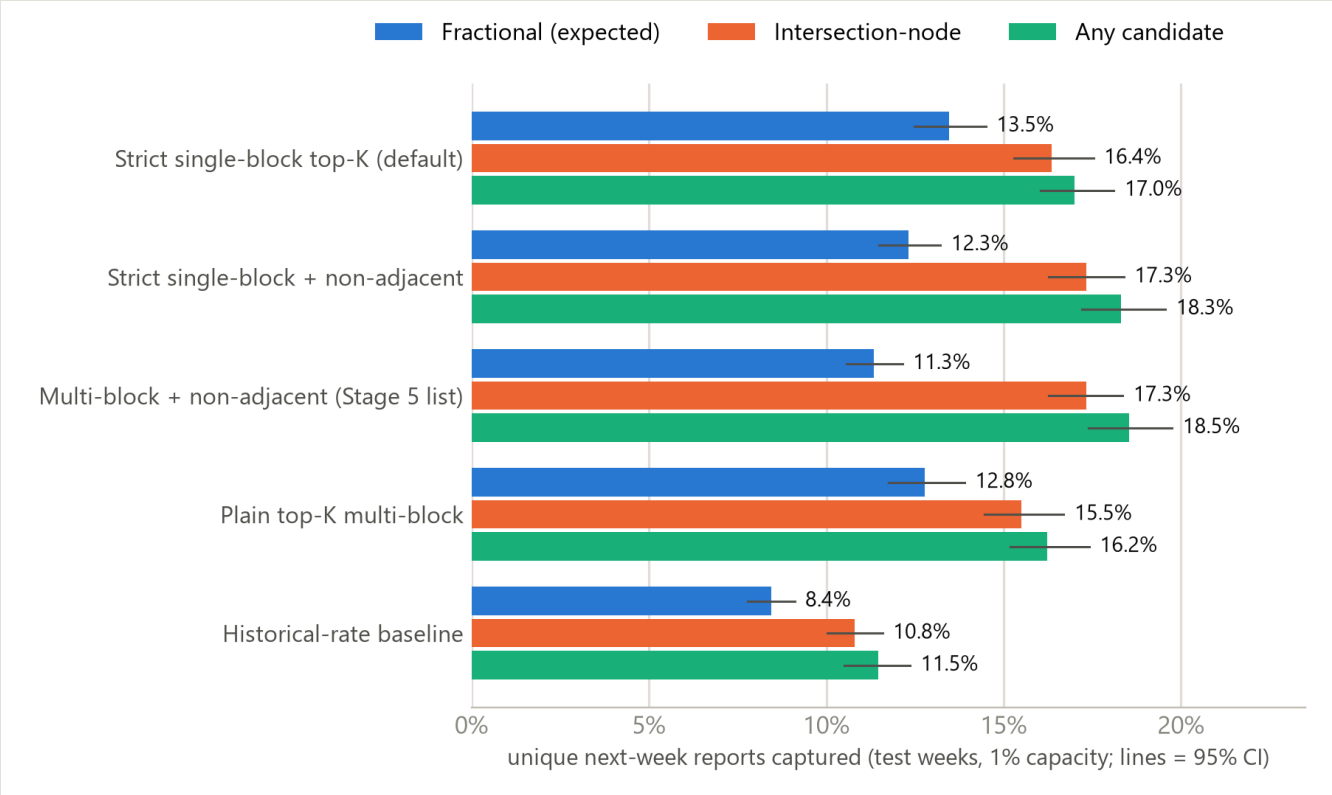


Figure 6. Capture under three definitions, test weeks, 1% capacity.

Why the ranking of lists flips. The non-adjacent rule never lists two blocks that meet at one intersection, so a single listed block "catches" an intersection report under the any-candidate definition.

Decomposition of the Stage 5 non-adjacent list against plain top-K (test, 1%, reports; `outputs/stage7_scratch_eval/decomposition_1pct.csv`):

Report type	Any candidate	Assigned block	Fractional
One candidate block	+84	+84	+84
Ambiguous intersection	+275	−321	−291
Ambiguous blockface / other	+30	−40	−34
Total	**+389**	**−277**	**−241**

Conclusions:

- The non-adjacent lists are better only under the end-intersection coverage assumption. Under expected capture the Stage 5 list is −1.4 pp against plain top-K (paired 95% CI −2.1 to −0.9 pp).
- The strict top-K list is best under expected capture and never worse than plain top-K under any definition: test fractional +0.2 to +1.1 pp.
- **Disclosure:** the non-adjacent rule was written after test-period top-K results existed. Its validation "confirmation" was therefore not independent.

10. The March 2026 surge and weekly volume forecasts

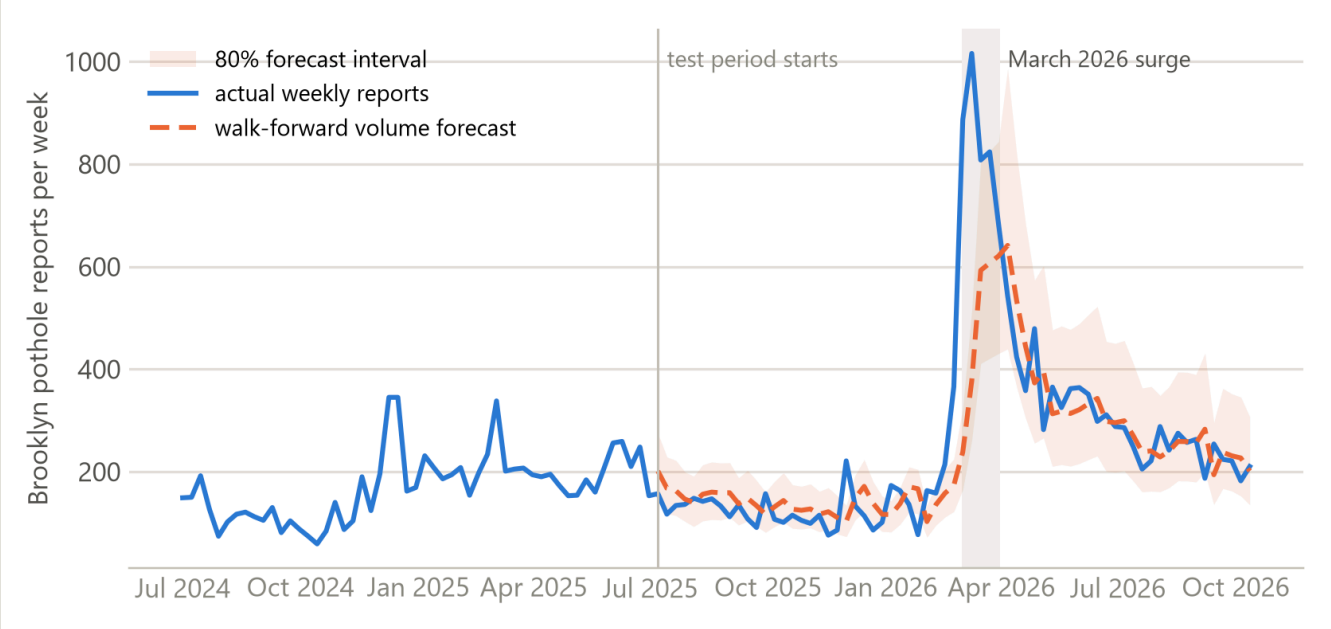


Figure 7. Weekly Brooklyn pothole reports and the walk-forward volume forecast with its 80% interval (`outputs/stage5/volume_forecast_results.json`).

[Implemented] The weekly volume model is a Poisson GLM. Validation chose its *recent* feature set (alpha 1.0); a weather feature set was tested and not selected. It is shown in the dashboard as context only.

[Measured] Results by period:

Period	Forecast MAE	"Same as last week" MAE	4-week mean MAE	80% interval coverage	Mean actual reports/week
All 67 test weeks	64.2	64.1	75.0	76%	256
Excluding March 2026	40.6	45.5	51.1	79%	209
March 2026 (5 weeks)	355.9	295.2	371.5	40%	843

- **Overall:** the forecast does not reliably beat simple baselines. The MAE difference against the 4-week mean has a 95% CI of −24.2 to +1.4.
- **March 2026:** reports ran about 4× normal, and the forecast fell **42%** short.
- **Ranking in the surge:** the strict top-K list still captured 16.9% vs 13.1% (any candidate) and 12.9% vs 9.2% (expected) against the historical rate. This rests on only 5 weeks, so it is directional.
- **Calibration in the surge:** 0.35 (strict model).

[Measured] A pre-specified rule that raised capacity when the volume forecast was high was also tested, on the Stage 5 list:

- **Adaptive:** 4,041 reports from 23,701 inspections, 17.0 per 100.
- **Fixed 1%:** 3,125 reports from 15,745 inspections, 19.8 per 100.

The adaptive rule lowers yield per inspection, so it is not recommended.

11. Real-life operation

How a weekly cycle would work **[Proposed]**. The software parts are implemented; the operating process is not.

1. **Monday:** new 311 data (lagged 2 days) is ingested and validated; features are built point-in-time.
2. **Scoring:** the current model scores all 23,448 blocks. A supervisor chooses a capacity, and RoadWatch produces the ranked worklist with reasons.
3. **Hand-off:** the supervisor reviews the list on the map, removes blocks with known works, and exports CSV or GeoJSON to the crew's GIS or routing tool.
4. **Inspection:** inspectors visit each block and record what they actually find. RoadWatch's recommendation is *inspect*, never *repair*.
5. **Retraining:** monthly, as in the walk-forward test, under governance: the frozen selection rule, drift monitoring of predicted versus observed volume, and no tuning on evaluation weeks.

Evidence needed before deployment (§12):

- inspection outcomes;
- field confirmation of what an inspection covers;
- a prospective shadow trial;
- an equity review;
- cost data.

12. Improvement roadmap [Proposed]

None of these has been built or measured; they are kept separate from the demonstrated results above.

Improvement	Data required	Proposed implementation	Evaluation metric
1. Confirmed inspection outcomes and severity	Per inspected block: date, defect found (yes/no), number and severity (depth or area class), repair date	Add outcome columns to the exported worklist (keyed by `block_id`, `forecast_week_start`); a weekly import joins them back; a new `defect_found` target trained with the same walk-forward protocol once enough weeks exist	Defects found per 100 inspections and severity-weighted capture, against the historical-rate list on the same weeks (paired CI)
2. Location resolution at intersections	GPS or photo coordinates with each 311 report; inspector GPS at the defect	Snap coordinates to the nearest block leg within a tolerance; for legacy intersection reports, learn from inspection outcomes which leg usually holds the defect	Share of reports on one block (now 59.2%); the gap between any-candidate and fractional capture should shrink
3. Reliable publication timestamps	Daily API snapshots recording `:created_at` and `:updated_at`	Schedule the existing `ingest` daily (immutable snapshots already supported); measure each record's availability lag; replace the assumed 2-day lag with the observed distribution	Lag distribution; backtest with true as-of data against the assumed-lag backtest
4. Local traffic, weather and road condition	Automated traffic counts, more weather stations, recent pavement ratings and work orders (the last two are already ingested)	Add as point-in-time features with measured lags; select only on validation weeks; walk-forward test	Paired change in precision@1% and AP; feature ablation

Improvement	Data required	Proposed implementation	Evaluation metric
5. Surge handling and calibration	Recent weekly report totals (already available)	Recalibrate weekly on the latest matured weeks; regime features from Brooklyn-wide volume; alert when predicted/observed volume leaves a band	Monthly calibration ratio near 1; Brier score; interval coverage in surge months
6. Reporting-equity monitoring and coverage	Community-board or ZIP boundaries; independent condition data (inspector findings, pavement ratings); demographics for monitoring only, never as model inputs	Dashboard panel comparing inspection share with block or length share per area; optional minimum coverage per area (the ZIP cap exists as `--diversify`)	Inspections and defects found per area; capture cost of any floor
7. Weather in weekly volume forecasts	NOAA daily history (ingested), weather forecasts issued before *t*, freeze–thaw counts	Extend the already-tested weather feature set of the Poisson GLM with forecast weather; walk-forward evaluation	MAE against "same as last week" with a paired CI; interval coverage in surges
8. Repair effectiveness and cost	Repair cost and crew time per job, travel, re-report rates after repair, claims data	Randomised or alternating-week trial comparing RoadWatch lists with current practice	Cost per verified defect, time from report to repair, re-report rate. Only then make any savings claim

13. Reproduction, environment, testing and troubleshooting

13.1 Environment

Item	Value
Python	3.12 in the project-local `.venv` (created with `uv`)
Pinned dependencies	`pyproject.toml`; full lock in `requirements-lock.txt`
Optional extras	`presentation` (python-pptx, markdown, pypdfium2) for rebuilding the final documents; `audit` for the Stage 1 scripts

Fresh install (PowerShell, repository root):

```
uv venv .venv --python 3.12
uv pip install --python .venv\Scripts\python.exe -r requirements-lock.txt
uv pip install --python .venv\Scripts\python.exe --no-deps -e .
```

13.2 Reproduce the results

```
$py = ".venv\Scripts\python.exe"
& $py -I -m roadwatch rebuild all              # processed tables from archived snapshots (offline)
& $py -I -m roadwatch stage3                   # blocks, matching, labels
& $py -I -m roadwatch stage4 features          # point-in-time features
# do NOT re-run "stage4 select": data/processed/stage4/selection.json is the frozen record
& $py -I -m roadwatch stage4 test              # walk-forward (skips experiments already predicted)
& $py -I -m roadwatch stage4 evaluate
& $py -I -m roadwatch stage4 uncertainty
& $py -I -m roadwatch stage5 capture-definitions
& $py -I -m roadwatch stage5 alternative-worklists
& $py -I -m roadwatch stage6 build
& $py -m streamlit run app.py                  # dashboard at http://localhost:8501
```

Independent check:

```
& $py -I -X utf8 scripts\audit\stage7_reproduce.py . outputs\stage7_repro_root
```

Strict top-K worklist for one week from the command line:

```
& $py -I -m roadwatch stage5 worklist --week 2026-03-16 --capacity 0.01 --policy primary__LGBM --
rule topk
```

13.3 Tests

`& $py -I -m pytest -q` runs the full suite:

- **Unit tests** use labelled synthetic fixtures: matching, labels, leakage cut-offs, walk-forward windows, selection rules, capture definitions and exports.
- **Integration tests** read the real saved outputs, and are skipped if those outputs are absent: dashboard values verbatim from saved files, default list = strict top-K, and downloads equal to the saved selections.

Final deliverables: `scripts/final/verify_final.py` re-derives every number in the pitch and the final documents from saved files.

13.4 Troubleshooting

Symptom	Cause	Fix
Dashboard shows "A required saved output is missing: …"	A stage output was not built	Run the command printed in the message (for example `stage6 build`)
`ingest` exits with code 1 or 2	A core (1) or optional (2) source failed validation or download	Check `outputs/logs/run_*.log`. The previous processed table is still in use. Re-run `ingest <source>`, which resumes
Socrata rate limits or timeouts	Anonymous access	Set `SOCRATA_APP_TOKEN` in `.env` (never commit it); retries with back-off are automatic

Symptom	Cause	Fix
Map shows block lines but no streets	No internet for CARTO basemap tiles	Expected offline; all other features work
`ModuleNotFoundError: roadwatch`	Package not installed in the active interpreter	Use `.venv\Scripts\python.exe`, or `uv pip install --no-deps -e .`
Numbers differ from this manual	Outputs were rebuilt with different inputs	Re-run `scripts\final\verify_final.py`, which lists every mismatch with its source file

14. References

Data sources (all accessed through their public APIs; snapshots retrieved 2026-10-24):

- NYC Open Data: 311 Service Requests from 2010 to Present, dataset `erm2-nwe9` (data.cityofnewyork.us).
- NYC Open Data: NYC Street Centerline (CSCL), dataset `inkn-q76z`.
- NYC Open Data: Street Pavement Rating `6yyb-pb25`; Street Pothole Work Orders – Closed `x9wy-ing4`; DOT resurfacing datasets `ffaf-8mrv` and `xnfm-u3k5`.
- NOAA National Centers for Environmental Information: Global Historical Climatology Network – Daily, station USW00094728 (New York Central Park), via the NCEI Access Data Service.

Software: Python 3.12; pandas; pyarrow; geopandas and shapely; LightGBM; scikit-learn; Streamlit and pydeck (CARTO basemap). Exact versions are in `requirements-lock.txt`.

Project documents:

- `docs/data_audit.md` (source audit);
- `docs/stage2_report.md` … `docs/stage6_report.md`;
- `docs/stage7_audit_report.md` (independent audit; authority for corrected findings);
- `docs/risks.md`;
- `docs/final/user_guide.md`.

15. Glossary

Term	Meaning
Block	A CSCL street segment between two intersections (`physicalid`); the unit RoadWatch ranks
Block-week	One block in one forecast week; the unit of labels and predictions
Candidate block	A block a report could refer to; ambiguous reports have several
Strict single-block target	Each report counts on exactly one block (unique match or deterministic hash pick)
Multi-block target	Every candidate block of a report counts as positive
Unknown label	A block-week excluded because too many reports that week were unlocatable; never treated as zero
Point-in-time feature	A feature computed only from data published before the forecast date
Availability lag	Delay between an event and its publication (311: 2 days; NOAA: 5 days)
Walk-forward backtest	Repeatedly retraining on the past and predicting the following period, as if in real time

Term	Meaning
Holdout	The most recent 13 matured weeks before a retrain; used for early stopping and calibration
Calibration in the large	Predicted ÷ observed number of positives
Precision@1%	Share of the top 1% (235) listed blocks that received a report
Report capture	Share of next-week reports falling on listed blocks (see the four definitions in §9)
Paired bootstrap CI	Uncertainty interval from resampling the same test weeks for two methods
Top-K	Take the K highest-ranked blocks
Non-adjacent rule	Skip a block that shares an intersection node with a block already selected
Capacity scenario	Weekly number of inspections: 0.5%, 1% or 2% of blocks, or custom; experimental, not a budget
Historical-rate baseline	Rank by the share of past weeks with a report
MAE	Mean absolute error (reports per week)
Snapshot / manifest	Immutable copy of an API download / the record of its checksums and validation